DAY 28 / 90

AWS Week — Day 6: Amazon EKS

Kubernetes on AWS • Cluster • Node groups • IAM • Deployment

Goal: Understand how Amazon EKS runs Kubernetes on AWS, deploy a small app, expose it through a load balancer if budget allows, then remove lab resources.

Source mapping: Week 3 AWS source document — Day 6: EKS. Challenge Day 28/90.

Scope rule: Do not repeat Days 22–27 (AWS overview, IAM, S3/CloudFront, EC2 setup, RDS or VPC design). Focus on EKS, node groups, Kubernetes deployment on AWS, and cleanup.

Cost warning: EKS control plane, EC2 worker nodes, load balancers, storage, NAT Gateways and public IPv4 addresses may cost money. Check current pricing before creating resources. A budget alert is not a hard spending cap. Use the design-only path if unsure, and delete all lab resources after testing.

1. Key concepts

Term	Simple explanation
Amazon EKS	AWS-managed Kubernetes service. AWS manages the control plane; you manage workloads and worker capacity.
Control plane	Kubernetes API and coordination components managed by AWS in EKS.
Worker nodes	Compute machines, often EC2 instances, where application Pods run.
Managed node group	Group of EC2 worker nodes integrated with EKS management.
kubectl	CLI used to inspect and manage Kubernetes resources.
IAM access	Controls who can access or administer the EKS cluster.
IRSA	IAM Roles for Service Accounts; gives workloads scoped AWS permissions through a Kubernetes service account.
AWS Load Balancer Controller	Can create AWS load balancers from supported Kubernetes resources.
EBS/EFS	AWS storage choices for persistent workloads, depending on access requirements.
Autoscaling	Pod autoscaling changes replicas; node autoscaling changes available compute capacity.

2. Architecture

```
You / kubectl
|
v
EKS control plane (AWS-managed)
|
v
Worker node group (EC2) ---> Pods / application
|
v
Service / LoadBalancer ---> Users
```

An Active cluster does not guarantee that the app is reachable. Nodes, Pods, Service/load balancer, network routes and security rules must all work.

3. Task A — Prepare safely

• Check current EKS, EC2, load balancer, storage and networking pricing before starting.

• Choose a cluster name, region, supported Kubernetes version, VPC/subnets and cleanup time.

• Use least-privilege IAM permissions; do not share long-lived access keys.

• If you cannot risk AWS charges, skip cluster creation and complete the architecture, manifest and interview tasks below.

4. Task B — Create EKS (optional paid lab)

• Using the AWS Console or an approved eksctl/CLI workflow, create an EKS cluster with a suitable VPC/subnet configuration.

• Wait until the cluster status is Active, then create a managed node group and wait for its nodes to become Ready.

• Configure kubectl credentials for your chosen cluster and region. Exact commands depend on your setup.

```
aws eks update-kubeconfig --region <REGION> --name <CLUSTER_NAME>
kubectl config current-context
kubectl get nodes
kubectl get namespaces
```

Verify the context before applying manifests. Never publish account IDs, credentials or kubeconfig contents.

Design-only path: Do not create an EKS cluster. Draw the architecture and explain how the control plane, node group and workload connect.

5. Task C — Deploy a small Nginx app

Save this as **deployment.yaml**. Use a LoadBalancer Service only if you have checked costs and intentionally want to provision an AWS load balancer.

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: nginx-demo
spec:
replicas: 2
selector:
matchLabels:
app: nginx-demo
template:
metadata:
labels:
app: nginx-demo
spec:
containers:
- name: nginx
image: nginx:stable
ports:
- containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
name: nginx-demo
spec:
type: LoadBalancer
selector:
app: nginx-demo
ports:
- port: 80
targetPort: 80
```

```
kubectl apply -f deployment.yaml
kubectl get deployments
kubectl get pods -o wide
kubectl get services
```

• Wait until the two Pods are Ready.

• If a load balancer is provisioned, open its external address and verify the Nginx welcome page.

• Record Pod status, node placement, Service type and external address.

• For the no-load-balancer route, change Service type to ClusterIP and do not expect an external address.

6. Troubleshooting

Symptom	First checks
kubectl cannot connect	Check kubeconfig context, region, cluster name, IAM access and cluster status.
Nodes NotReady / missing	Check node group status, EC2 capacity, node IAM role and networking.
Pod Pending	Check capacity, scheduling events, resource requests, taints/tolerations and storage requirements.
ImagePullBackOff	Check image name/tag, registry access and Pod events.
LoadBalancer has no address	Check Service events, subnet/network configuration, IAM permissions and controller requirements.
App times out	Check Pod readiness, Service endpoints, load balancer health, security rules and application port.

7. Concepts to explain

• How is IAM access to EKS different from an application Pod needing AWS permissions?

• Explain IRSA and why workloads should receive only the permissions they need.

• Compare EBS and EFS at a high level; select storage based on workload and access needs.

• Explain why Pod autoscaling and node autoscaling solve different problems.

• Describe the purpose of the AWS Load Balancer Controller.

8. Cleanup — mandatory

• Delete Kubernetes resources: kubectl delete -f deployment.yaml.

• Delete the EKS managed node group, then delete the EKS cluster using the workflow used to create it.

• Check for leftover load balancers, target groups, NAT Gateways, Elastic IPs, EBS volumes and other lab-created resources.

• Inspect and delete only CloudFormation stacks belonging to this lab, if created by your tool.

• Verify the cluster, node group and lab load balancer are gone. Do not delete shared resources from previous days.

• Review billing later; some usage may appear after deletion.

Important: Deleting the Deployment alone is not enough. EKS, EC2 nodes and load balancers can continue to incur charges.

9. Deliverable

• Create **day6-eks/** with deployment.yaml and a README describing architecture and steps.

• Include a diagram: kubectl → EKS control plane → node group → Pods → Service/load balancer.

• Add notes about IAM access, IRSA, EBS/EFS and autoscaling.

• Write one troubleshooting case (real or clearly labelled simulated), plus cleanup and cost notes.

Only 3–5 screenshots: cluster status; kubectl get nodes; Pods/Deployment Ready; Service/load balancer and Nginx page if created; cleanup proof. Hide account IDs and credentials.

10. Interview questions

Q. What is Amazon EKS?
A managed Kubernetes service. AWS manages the control plane; you manage workloads and worker capacity.

Q. What is a managed node group?
A group of EC2 worker nodes integrated with EKS management.

Q. What is IRSA?
It gives Kubernetes workloads scoped AWS permissions through a service account and IAM role.

Q. Pod vs node?
A Pod runs one or more containers; a node is the worker machine that runs Pods.

Q. Why might a Pod be Pending?
Insufficient resources, scheduling constraints, taints/tolerations or storage/network requirements.

Q. Why use a load balancer?
It provides a stable entry point and routes traffic to an application.

Q. Pod autoscaling vs node autoscaling?
Pod autoscaling changes replicas; node autoscaling changes available compute capacity.

Q. Why clean up EKS labs?
Control plane, nodes, load balancers, storage and networking resources can keep costing money.

11. Completion checklist

• I can explain EKS control plane and worker nodes.

• I deployed or designed a Deployment and Service for EKS.

• I understand IAM access, IRSA, EBS/EFS and autoscaling basics.

• I documented one troubleshooting scenario and 3–5 useful screenshots.

• I deleted lab resources and verified that chargeable dependencies were removed.

Learn → Build → Break → Fix → Explain